

Acadia: Enhancing CXL Memory Pods via Sparse Topology

Submission #1166

Abstract

The Compute Express Link (CXL) interconnect enables compute “pods” with memory pooling across hosts to reduce cost and improve efficiency. However, existing pod designs are small or require indirection through expensive switches. These conventional designs implicitly assume that pods must fully connect all hosts to all CXL pooling devices.

This paper breaks with this conventional wisdom to create “Acadia” pods. Acadia connects each host to a bounded number of small multi-port pooling devices (e.g., 8), and each pooling device connects to different subsets of hosts, forming a sparse topology of hosts and pooling devices. Despite no longer having a global memory pool, we show that Acadia pods still effectively support memory pooling and useful data sharing and communication patterns. Relative to conventional pods, Acadia is more cost-effective (using near-commodity pooling devices) and enables larger pods (allowing more pooling flexibility and greater communication reach).

We build a three-host CXL pod prototype and simulate scaled pods with 96 hosts under measured device characteristics and physical constraints (1.5 m copper cables). On hardware, Acadia RPCs are 3.3 $\times$ faster than in-rack RDMA and 2.4 $\times$ faster than CXL switches. In simulation, Acadia’s sparse topologies achieve net server cost savings of 3-5.4% whereas CXL switches are net negative cost.

1 Introduction

General-purpose cloud systems face a growing memory shortfall. Compute scales with die area (n^2), but the DRAM a socket can attach over DDR4/5 is “beachfront”-limited ($4n$). As core counts and per-core performance have driven per-socket compute up by 5–10 $\times$ over the past decade, socket-attached DRAM capacity has grown only 2–3 $\times$ [34, 66, 90]. Right-sizing cores or pushing more work to scale-out helps, but the most cost-effective lever is: *attach more memory to the same socket*.

CXL memory expansion is the first step. Compute Express Link (CXL) provides a practical path today to add capacity with CPU load/store semantics [30]. CXL-attached DRAM (“expansion”) is accessed like local memory (ld/st); it is supported by major CPU platforms [4, 9, 27] and shipping servers that double memory capacity [72, 102, 112, 116, 118].

⁰Acadia is known for its unique hydrological system, where ponds are sparsely connected through streams and wetlands, forming a cohesive yet sparse network.

While expansion adds latency (~ 260 ns vs 115 ns), many web, key-value, analytics, and database workloads tolerate this NUMA-like penalty [6, 79, 136].

This paper explores the next step of creating a CXL “pod” spanning multiple hosts and using CXL memory modules as a “scale-up” interconnect. Pods enable memory pooling, which is known to reduce DRAM spend (often half of server cost) by multiplexing capacity demand across hosts [6, 34, 67, 77, 126, 132]. Additionally, the intra-pod CXL fabric also happens to create a low-latency data sharing and communication fabric that, compared to switched Ethernet, improves performance of several workload classes. Specifically, pod-wide RPCs, coordination, and collective-based data movement can be sped up by sharing memory buffers over the CXL fabric [5, 13, 81, 83, 115, 134, 135]. Realizing these promises, however, largely depends on performance and topology of the CXL fabric.

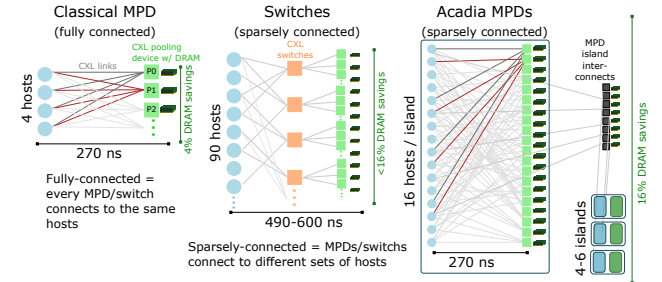


Figure 1: Acadia combines offering low latency for pooling memory and communication use cases, while offering a large pooling domain for multiplexing memory demands.

Two practical ways exist to connect hosts to pooled memory. *Multi-ported devices (MPDs)* collapse a tiny on-device interconnect with the memory controllers so multiple hosts can attach directly, *without a CXL switch* [32, 42, 44, 50, 77, 82, 125]. Today’s MPDs are small (2–4 ports; ~ 270 ns access) [20, 32, 47, 63, 84, 103, 129], and thus thought to limit pods to a handful of hosts; prior work assumes *fully connected* pods in which every MPD connects to every host [32, 42, 44, 50, 77, 82, 125]. *CXL switches* expand connectivity (dozens of ports), but add (de)serialization hops and optics, raising latency and CapEx [14, 30, 76]. In practice, three pressures collide: (1) build a pod large enough to pool effectively; (2) keep common access/communication paths short and diverse enough for other use cases; and (3) keep pod CapEx low enough that the savings exceed the cost of the fabric.

We keep the simplicity of MPD-based pods but break

from prior work by considering *sparsely-connected* hosts and MPDs. In a sparsely-connected topology, each host connects to multiple MPDs, which in turn connect to *different sets of hosts*. This allows pods to scale beyond the MPD port count, e.g., dozens of hosts with 4-port MPDs, without the expense and latency of switches [76]. While sparse topologies take inspiration from expander-like topologies (e.g., Jellyfish/Xpander) [46, 109, 123], we need to adapt this concept to *switchless, host-MPD bipartite* graphs under port and cabling constraints. Our study of pooling and data sharing applications reveals several key graph properties that cannot be satisfied by classical or expander-like topologies. Specifically, we observe the need to create large pods and maximize the number of reachable MPDs for pooling efficiency while also achieving low diameters for low-latency communication, which disqualify expanders and randomness. Instead, we use combinatorial design to construct topologies with the exact properties required for effective CXL pooling and sharing.

Acadia is our design for near-term, cost-conscious CXL pods built from *sparse MPD* topologies. We bound scope to what is deployable today: *no inter-host hardware coherence*, copper-first wiring, realistic port budgets, and minimal forwarding by hosts. The resulting pod topologies combine multiple *islands* of highly-connected servers, each of which has low-latency intra-island connections, with many inter-island connections to realize the benefits of memory pooling in large pods.

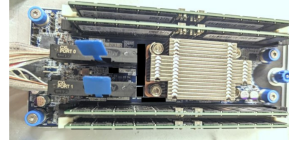
We implement a minimal prototype of Acadia on real hardware spanning three hosts and three MPDs. Acadia matches the performance of existing CXL memory expansion systems, where a CXL switch would otherwise increase the slowdown of 25% of applications by $3\times$. Acadia’s communication latency is $3.2\times$ lower than in-rack RDMA, $2.4\times$ lower than a CXL switch, and $4.5\times$ lower than of other sparse topologies with good pooling savings.

We simulate scaled-up Acadia topologies of 25, 64, and 96 hosts based on 4-port MPDs. Figure 1 shows how Acadia achieves similarly low latency to fully-connected MPD topologies from the CXL literature [77]. Acadia’s savings from poolings alone can pay for the fabric both in cases with existing CXL expansion (5.4% overall cost reduction) and without CXL in place (3% overall cost reduction). CXL switches always cost more, even after factoring their memory cost savings.

While CXL can outperform Ethernet and RDMA, it does not replace it. As with GPU pods, we expect inter-pod communication to still use Ethernet as its scale-out network. Unlike GPU pods which are built to maximize performance at high cost, CXL use cases are often cost sensitive. Our work thus shows how enable multiple use cases to leverage the intra-pod CXL fabric to make the investment worthwhile.

Contributions.

- First paper to question the assumption that CXL pods must fully-connect hosts and pooling devices



Device	P50
CXL expansion	250–270 ns
CXL 2/4-port MPD	270–300 ns
CXL switch	490–600 ns
RDMA ToR	3550 ns

Figure 2: Left: example two-port CXL device. Right: End-to-end read latency (P50) on random 64-byte cachelines accessed via different CXL devices and RDMA on Intel Xeon 6 and AMD Turin.

- Systematize practical CXL pod constraints and objectives.
- Show fully-connected is unnecessary for large CXL use cases
- Evaluate shared-memory communication on real sparse MPD hardware. Unify performance/\$ via a cross-topology cost model.

2 State of CXL in 2025

This section describes how (a) CPUs access CXL-attached memory, and (b) how CXL can act as a scale-up intra-pod network.

The Compute Express Link (CXL) interconnect standard itself is described in detail in a recent tutorial [30]. We discuss real devices and constraints when using CXL.mem today. When a CPU issues a `ld` to an address associated with a CXL device and CPU caches do not have the data, the CPU sends a CXL.mem read *flit* to the device. CXL.mem flits use PCIe physical lanes and electrical signaling but implement custom low-latency protocol layers. The device reads the cache line from its (DDR4 or DDR5) DRAM and sends it back to the CPU, which places it into processor caches and the core.

On Intel Xeon 6 platforms a local DDR5 read takes about 115 ns. Reading from a good CXL.mem expansion device takes 200 - 300 ns. Using a CXL bus analyzer we measure that 75 - 170 ns of this latency is due to the CPU (most of the variability), 65 ns is due to CPU port round-trips and flight time, 25 ns is due device-internal processing, and 35-40 ns is due to DRAM access. On AMD Turin platforms, performance is similar. Devices typically offer multiple DDR4 or DDR5 DRAM channels.

A single CPU typically has multiple CXL ports. For example, Intel Xeon 6 production platforms use 64 CXL lanes per CPU socket, e.g., [55, 59, 71, 96, 117]. A single $\times 8$ CXL port offers 25-30 GB/s of read-only bandwidth, and twice that for $\times 16$. The CPU socket’s lanes are configurable as four $\times 16$ CXL or eight $\times 8$ CXL ports. In aggregate, a CPU socket can thus read from CXL devices at 200-240 GB/s.

There are three CXL.mem device types. An expansion device offers a single CXL port and exposes its memory to a single CPU. A multi-ported device (MPD) enables N CPUs to connect to the same controller and access its memory. These

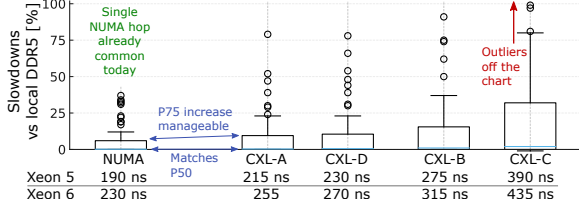


Figure 3: Box plots of workload slowdowns under different CXL latencies show that an increasing fraction of workloads sees slowdown around 390 ns on Xeon 5, which is equivalent to 435 ns on Xeon 6.

devices are readily available today and sometimes both options are available on the same device. Typical $N = 2$ examples offer two $\times 8$ ports, like Figure 2, including devices by AsteraLabs [63], Marvell [84], Google and Meta’s Hyperscale CXL [20], and xFusion [47]. Seagate and SKH have $N = 4$ prototypes [32, 103, 110] with detailed performance benchmarks on Seagate [129]. On our lab system, our $N = 2$ MPD offers 270 ns read latency and 28 GB/s read bandwidth per port.

CXL switches offer up to 32 CXL ports and can forward CXL packets between them, e.g., the XC50256 switch offered by XConn [130]. For every flit round-trip, a switch must deserialize and reserialize the flit twice, which adds at least 220 ns of latency [53] with actual measurements somewhat higher (see table in Figure 2). In today’s CXL 2.0 standard, switches connect to hosts and devices, but cannot connect to other switches.

CXL devices connect to the CPU’s CXL ports with physical PCIe5 pins and cables. PCIe5 systems are constrained by insertion loss, which admits a simplified analysis. Specifically, we have an insertion loss budget of 36 dB at 16 GHz. The CPU package, motherboard, MPD board, and MPD chip leave only around 10 dB of margin. Accounting for edge connectors, this constrains cable lengths to less than 150 cm. This matches the results from prior analysis [99, 120]. Longer copper cable would require a retimer, which adds latency and power consumption [10], or optical cables, which also add power and are expensive [1].

Hosts can concurrently access the memory behind a MPD or behind a switch. CXL 2.0 does not support hardware coherence, so software must ensure coherence [30].

3 Modeling CXL Device Cost

To understand the cost tradeoffs of different CXL pooling topologies, we model the cost of CXL devices and cables. As is common with datacenter server components, vendor prices are subject to non-disclosure agreements [12]. We thus rely on a die area model that captures typical IO-pad limitations [111].

Our model is grounded with three data points. First, use

a 5nm design for a MPD with eight $\times 8$ ports, and 8 DDR5 channels proposed by ARM [44]. Second, we have scaled-up die area estimates from the 6nm design of the AMD Zen4 (Genoa) IO Die [89] and we can estimate the die cost by interpolating the common factor of the IO die from the cost of CPUs with different core counts. Third, we have spoken to supply chain experts on pricing of expansion and 2-port devices today. We also model cable costs based on copper and assembly prices, as well as optical cable prices from a recent vendor demonstration [1].

Figure 4 shows our die area estimates, CXL device costs, and cable costs. The cheapest device is a single-ported ($\times 8$) expansion device with two DDR5 channels, at \$ 200. Compared the expansion, we provision MPDs with twice the number of CXL ports per DDR5 channel, and scale this linearly. This makes an $N = 2$ -port MPD slightly more expensive than an expansion device, which is pessimistic because we know that in practice many expansion devices actually use one $\times 16$ that can be bifurcated into two $\times 8$ ports. A $N = 4$ MPD amortizes some of the fixed area overheads, but we assume that vendors would charge a slightly higher markup. At $N = 8$, MPDs are IO-pad limited and prices increase significantly in our model. The same applies to switches with 24 or 32 ports, which are significantly more expensive than MPDs.

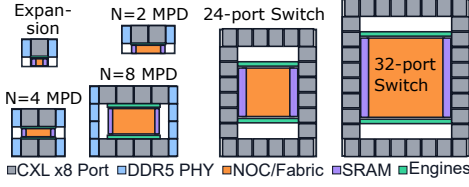
Back-of-the-envelope cost calculations show why MPDs can be a cost-effective building block for CXL pods, while switches can be too expensive. Say 96GB DDR5-5600 mainstream DIMMs cost \$432 (\$4.50/GB as a common assumption [51]). Each host uses 8 CXL expansion devices which are 32 DIMMs assuming two DIMMs per channel. Switching to $N = 2$ MPDs increases CXL device cost overall by \$320, which less than the price of one DIMM. So, we’d have to save at least 3% in memory capacity by pooling. In contrast, switches would have to save much more because they strictly add cost as we still need to buy the expansion devices. For example, using a 24-port switch means using 12 ports for hosts and 12 ports for expansion devices. Every host still needs eight $\times 8$ ports, to match bandwidth, which means multiple switches are needed. Even just adding three switches exceeds the entire cost of the CXL-attached DRAM for a single host. We remark that 4.50/GB is a low estimate; depending on the DIMM size and vendors, prices can go up significantly, especially for 128GB DIMMs [33, 119].

4 CXL Use Cases

CXL offers many potential use cases. We briefly review memory capacity expansion and those use cases that work well within today’s CXL constraints (Section 2).

4.1 Memory capacity expansion

DDR5 scales poorly with modern sockets because it is a wide, parallel interface: every additional memory channel consumes



Type	CXL×8	DDR5	Area [mm ²]	Price [\$]	CXL Cable Length [m]	AWG	Price [\$]
Expansion	1	2	16	200	0.50	30	23
MPD	2	2	18	240	0.75	28	29
MPD	4	4	32	510	1.00	28	36
MPD	8	8	64	2,650	1.25	26	55
Switch	24	0	120	5,230	1.50	26	75
Switch	32	0	209	7,400			

Figure 4: Cost model for CXL devices and cables. Die area estimates for devices with different CXL and DDR5 configurations (left) are prices based on a yield and markup model (middle). Die area figures are simplified but based on a real 8-port MPD layout and modern IO dies. Cable prices are based on copper material prices and markup (right).

large bundles of pins, routing layers, and on-package area for PHYs/IOs. At the same time, each DIMM is capacity-limited and high-capacity parts carry steep \$/GiB premiums (e.g., 3d stacking with TSV [45, 73]).

Capacity per unit socket performance has decreased over time, as core counts and per-core performance have grown faster than the number of available channels. 2025 Intel platforms offer roughly one-third of the 2017 DRAM capacity per unit of socket performance. This is a structural trend and will continue on all platforms that use DDR memory. Other DRAM protocols (LPDDR and HBM) are even more capacity-limited.

CXL addresses these structural limits by using serial links and decoupling the memory controller from the CPU package. Serial CXL/PCIe lanes deliver 5-10× more capacity per pin than DDR, and placing controllers in CXL devices enables fan-out. Practically, CXL can 2-2.5× socket memory capacity today. For example, on the scalable Xeon 6 platforms, a socket supports eight DDR5 channels and 64 CXL lanes, leaving 64 lanes for PCIe devices. This allows connecting eight expansion devices each with two DDR5 channels (Section 3).

CXL memory expansion is latency sensitive. Figure 3 shows that application slowdowns on CXL significantly increase for devices at around 400 ns of latency, based on data from prior work [79]. Adapting applications [6, 67], OS tiering [31, 68, 105, 124, 131, 136], or hardware tiering [?, 20, 136] can only partially offset these slowdowns, especially at the tail. Thus both datacenters and workloads look for low-latency access to CXL memory.

4.2 CXL Capacity Pooling

Servers are typically provisioned with enough memory capacity to handle peak demands. Unfortunately, capacity demand is highly variable across time and servers which leads to a large gap between average demand and peak [77]. Pooling capacity aims to multiplex these per-server peaks so that capacity can be provisioned much closer to the peak, thus using less overall memory. Pooling CXL involves allocating CXL pod capacity to hosts for exclusive access and dynamically changing this allocation over time. It thus does not require any cross-host synchronization or coherence.

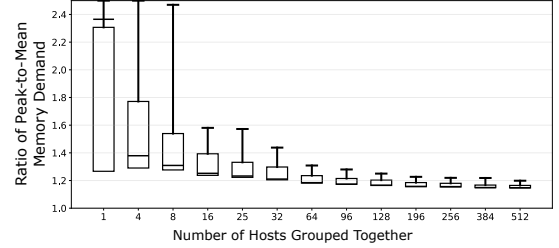


Figure 5: Ratio of peak memory demand to average memory demand across groups of hosts of different sizes show that we need to group large numbers of hosts to reduce outliers. Even groups of 25-32 hosts, we would still need about $1.5\times$ memory capacity to handle peaks. This data is for VM traces from a large cloud provider, gathered from ten production clusters over two weeks.

In CXL, dynamic allocation is implemented at a block size called extents [30, 77]. Importantly, memory is treated as a capacity resource, i.e., hosts do not typically care which extent is allocated to them. This enables significant flexibility in the pod memory allocation algorithm, which we further discuss in Section 5.

Prior work has shown significant savings from CXL capacity pooling and that these savings increase with the number of hosts in the pod. For example, we can reduce memory cost by multiplexing demands across cloud hosts [77, 126], lower cost and improve performance in in-memory databases [6, 67], lower cost for serverless function platforms [132], and improve application performance by adding more memory [39]. Figure 5 shows that the peak to average memory demand decays slowly when grouping hosts in sets of increasing sizes. We need to group large numbers of hosts to reduce outliers. This shows gains beyond prior work, in which savings increased from about 6-7% to about 12% of memory capacity when pooling between 4 to 16 hosts [77].

4.3 Practical Shared Memory Use cases

CXL today allows for data sharing among hosts but does not support cache coherence across hosts (Section 2). For this reason, CXL sharing is straightforward for read-only or

single-writer scenarios. In particular, CXL is effective when used to overcome the need to serialize large and complex data structures, as required by standard networking, or when requiring very low latency.

Overcoming serialization. A common use case for CXL pods is fast RPCs [13, 81, 83, 134, 135]. Prior work has shown that the RPC stack, including serialization and compression makes up about 6-10% of CPU time in typical RPC-heavy services, and up to 20% for small 1 KB payloads [49, 54, 81, 83, 95, 97, 104, 113, 134]. Regular RPCs also have high tail latencies due to queuing delays and CPU scheduling delays [81, 83, 134]. In CXL pods, serialization and compression overheads can be avoided by allocating pairwise CXL buffers with offset pointers.

Fail-over between hosts also benefits from avoiding serialization or writing data to storage. SAP has shown that CXL pooling can reduce restart time after a failure by 40-85% [6]. Broadly, fail-over use cases often rely on variants of primary-backup replication [23, 85, 91, 121, 135, 137] which can be implemented with a single writer.

Shuffle operations are widely-used in distributed analytics, and often cited as a key use case for CXL shared memory [11, 13, 41, 75, 121, 128]. For example, Memverge has shown $2.8\times$ shuffle speedups in a prototype [121]. Again this speedups can enable cost savings by reducing the number of nodes required to meet performance targets.

Low-latency communication. CXL can facilitate low-latency communication as we show in Section 6. This is especially useful for distributed applications that rely on small messages such as consensus variants Viewstamped Replication [78], Zookeeper [48], Raft [93], proposer and acceptor messaging in Paxos [65], and HotStuff [133]. CXL-based distributed transaction system proposals also involve a central coordinator, such as a transactional CXL controller [37]. Even complex communication patterns typically are implemented based on message-passing interfaces [22, 28, 80, 106].

Many use cases for high-availability scale-out happen at three to 16 host scale. MySQL recommends three to seven nodes [94], quorum-based replica sets in MongoDB often have three to seven members [87], in-memory/OLTP scale-out such as SAP HANA TDI deployments typically cap at *up to 16* worker nodes per cluster [100, 101], and cloud databases with bounded per-cluster compute like Amazon Aurora scale to 15 read replicas [7]. Redis clusters for production are often deployed as six nodes (three masters + three replicas) [98]. Bigtable clusters start at three nodes [38]. Primary-backup replication for databases [23, 85, 91, 121, 135, 137] also typically happens at this scale.

Larger scale-out systems are often driven by data analytics. This includes Azure SQL Database Hyperscale with up to 30 named read replicas per primary [86], Postgres sharding with Citus where dozens of workers are common [21], cloud

data warehouses like Amazon Redshift, which support 128 hosts per cluster [8], SAP HANA scale-out for analytics with up to 94 hosts [74]; and managed cloud databases that scale by adding compute nodes or read replicas—e.g., Azure SQL Database Hyperscale supports up to 30 named read replicas per primary [86]. Even larger systems can span hundreds to thousands of hosts, but these typically scale by adding more clusters rather than scaling a single cluster.

5 Acadia Design

Acadia builds on MPDs as a low latency and cost-effective building block for CXL pods. We seek to fulfill the following three goals.

- Effective memory pooling, which requires a large number of interconnected hosts,
- Data sharing and low-latency communication, which requires hosts that directly share an MPD,
- Practical deployment, which limits us to short copper cables, MPDs with $N = 4$ ports, and eight CXL links per host.

We describe the tension between these design goals and then move from graph representations to physical realizations and the software stack.

Central to Acadia’s feasibility is the assumption that every host provides $X > 1$ independent CXL ports, allowing each host to connect simultaneously to multiple MPDs. Current hardware commonly offers $X = 8$ ports per host (Section 2), though we demonstrate that $X = 4$ ports per host is sufficient to realize Acadia’s potential (Section 6). Multiple host ports enable each host to connect to multiple MPDs which in turn connect to different subsets of hosts. For simplicity and clarity, we assume single-socket servers¹ where each CPU constitutes a distinct host.

5.1 Tensions & Balancing Use Cases

We seek to create a topology that works for memory pooling as well as data sharing and low-latency communication use cases. A CXL pod topology can generally be represented as a graph where edges connect hosts to MPDs. Since there are no host to host edges, any two hosts are at least two hops away, traversing an MPD. The same is true for MPD to MPD connections.

Memory pooling. Pooling reduces overall memory capacity by multiplexing server demand peaks [77]. With $\frac{1}{2}-\frac{2}{3}$ of capacity on CXL (Section 4), our goal is to reduce the *peak*

¹ Acadia topologies also apply to multi-socket servers. In this case, we still seek to connect pairs of CPU sockets with each other. However, we incorporate the cache-coherent processor interconnection network to reduce required CXL links.

across MPDs. Our main insight is that pooling *does not* require a single monolithic pool accessed by all hosts. The key is to think about hosts allocating CXL capacity from MPDs over time, not as a static one-time assignment. Hosts observe the load on their connected MPDs and allocate from the least-loaded MPDs, and thus can balance load across MPDs. Thus, what matters is that when some servers run “hot,” their excess can spill into *enough distinct* MPDs so that no small subset of MPDs becomes the bottleneck.

Topology drives whether this happens. A bad topology risks concentrating many hot servers onto the same few MPDs, thus requiring MPDs to provision more memory. Under a fixed host port budget X , we cannot add more neighbors, so the lever is *reaching distinct MPDs*. Different servers’ X -port neighborhoods should overlap as little as possible. This neighborhood diversity lets hot server sets fan out over more distinct MPDs, pushing the required per-MPD capacity toward the global average.

We seek to maximize the number of distinct MPDs reachable from the worst-case hot server set, as demand patterns are not known in advance. For simplicity, we primarily consider the minimum number of distinct MPDs reachable from any server. This is a natural proxy for the worst-case hot server set, since a server with poor reachability can easily be part of a hot set. We visualize reachability for a 16-host topology in Figure 6; this topology is sub-optimal as server ports are wasted on redundant paths between any host and its reachable MPDs. Figure 7 shows how larger 96-host topologies with specific structures (discussed below) significantly increase reachability.

In general, pooling improves with reachability and thus favors three key graph properties. First, larger topologies always offer more distinct MPDs overall. Second, low-diameter topologies ensure that no server is “far away” from many MPDs. Third, high symmetry ensures that all servers have similar reachability.

Data sharing and communication. CXL is uniquely suited for serialization-free data sharing and low-latency small-request communication. This requires pairs of hosts to be able to issue loads/stores to the same MPD directly. We show in Section 6 that any forwarding through other hosts loses CXL’s unique advantages. Even forwarding through switches can significantly inflate latency (§6).

The ideal topology for sharing and communication is thus a MPD topology where *every pair of hosts shares at least one MPD*. Specifically, topologies should have diameter two for host-to-host paths. This is significantly more stringent than typical low-diameter requirements, since MPDs have just $N = 4$ ports. Naturally, this means that graphs must be dense and cannot be large. Fortunately, we observed in Section 4 that many data sharing and low-latency use cases happen within sets of three to 16 hosts. However, this use case is in tension with memory pooling, which benefits from larger graphs.

5.2 Logical Network Topology

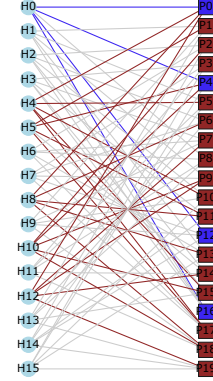


Figure 6: An Acadia island ensures that all host pairs share at least one MPD for data sharing use cases. For pooling, we largely care about the number of distinct MPDs reachable from any host (here, H0). The topology shows that any MPD can be reached in four hops, but there are many redundant paths which shows that this is suboptimal topology for MPD reachability and thus pooling.

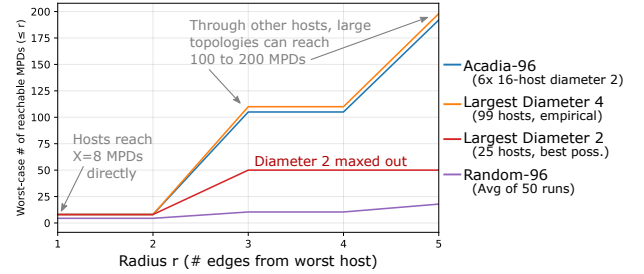


Figure 7: MPD reachability defines how well a memory demand hot spot can be dissipated across MPDs. A diameter 2 topology with $X = 8$ and $N = 4$ can only reach 50 MPDs within 3 edges from any host, while larger topologies can reach almost 100 MPDs at the same distance and continue to grow.

We observe that topologies with a diameter of two, when designed for optimizing data sharing, are small compared to the scale required for effective pooling. Acadia addresses this tension by creating *multiple diameter-two islands* and interconnecting them into a large pod for pooling.

Given typical values of $X = 8$ host ports and $N = 4$ MPD ports, a diameter-two topology can span maximally 25 hosts. Every host must reach any other host node after traversing exactly two edges, first an edge to an MPD and then an edge to the target host node. In general, any one host (say host A) connects to X different MPDs. Every MPD, has N ports, and one is already taken by host A. Thus, every MPD connects host A to $(N - 1)$ other hosts. So, host A shares MPDs with $X \times (N - 1)$ other hosts. After adding host A itself into the

pod size count, we get a pod size of $H = 1 + X \times (N - 1)$. For $X = 8$ and $N = 4$, this leads to $H = 25$.

A pod size of 25 hosts is insufficient for pooling. As shown in Figure 5, 25 hosts still have significant variability in demand. As shown in Figure 7, the worst-case reachability of a 25-host diameter-two topology maxes out at 50 MPDs at $r = 3$ edges from any host. We contrast this with the largest known diameter-four topology which spans 99 hosts². In the 99-host topology, the worst-case host can access nearly 100 MPDs at $r = 3$ and 200 MPDs at $r = 5$ edges.

The key insight of Acadia is to combine multiple islands of diameter-two into a large pod that has high reachability. Each island effectively supports data sharing and low-latency communication. A subset of X_i of host ports are dedicated to connect to a set of MPDs that are again dedicated to that island. Dedicating MPDs also ensures that different workloads scheduled on different islands do not interfere with each other. We interconnect islands using the remaining $X - X_i$ host ports and an additional set of dedicated interconnection MPDs.

One may expect that Acadia’s hierarchical construction sacrifices reachability. However, as shown in Figure 7, the reachability of a 96-host Acadia topology almost matches the best known 99-host diameter-four topology, which is the best topology we’ve found. Acadia does sacrifice the size of the data sharing-optimized pod to reach this reachability. Specifically, if $X_i = 8$, the pod size would be 25 hosts. We empirically find that $X_i = 5$ behaves well for reachability, which leads to islands of 16 hosts, a 40% lower data sharing scope. However, we observe that this is a reasonable tradeoff given the matching cluster sizes of common data sharing and low-latency use cases (Section 4).

Acadia topologies form a family of designs based on the number of islands. Assuming $X_i = 5$, we recommend pods of 25 (single island), 64 (four islands), 96 (six islands), and 128 (eight islands) hosts. We next discuss in more detail how to construct and interconnect islands.

5.2.1 Island Construction

Each island is a maximally-sized diameter-two topology. Constructing these topologies is a hard combinatorial design problem. Besides matching the diameter, maximal pod size requires two hosts that connect to the same MPD cannot both connect to another MPD. This would waste MPD ports that could otherwise connect to more hosts. We illustrate an example of a maximal diameter-two topology in Figure 6. Given $N = 4$ and $X = 5$, the pod size spans 16 hosts. This topology is what’s used in Acadia’s islands when $X_i = 5$.

We formalize the construction of a minimally-connected Acadia topology as a balanced incomplete block design

²There are no known constructions for diameter-four bipartite graphs. We use an expensive stochastic search that combines multiple construction approach and hill climbing. 99 hosts is the largest we found for $X = 8$ and $N = 4$ after 10 days of searching on a large compute cluster.

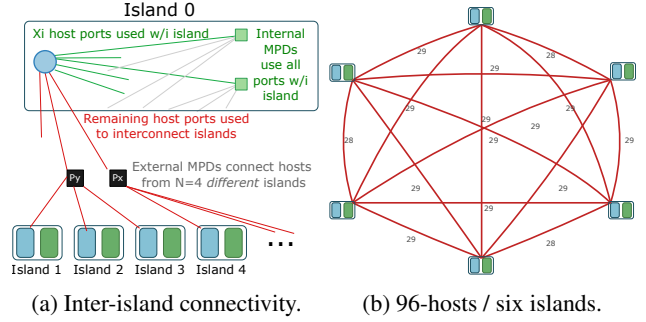


Figure 8: High-level view of an Acadia topology with multiple islands. (a) shows how X_i host ports are allocated to internal MPDs to form an island for data sharing. The remaining host ports (e.g., 3) connect to external MPDs that interconnect islands for pooling. (b) shows a six-island Acadia topology with 96 hosts at a high level. The numbers show the number of MPDs that each pair of islands share, we optimize for symmetry but this is not always mathematically possible.

(BIBD) [19, 35, 107]. BIBD is a classical combinatorial design for statistical experiments [19, 35, 107]. Specifically, a minimally-connected Acadia topology is a $2 - (H, N, 1)$ BIBD. Importantly, $\lambda = 1$ which defines the constraint on pairs of hosts connecting to exactly one common MPD. Constructing BIBDs is NP hard [26] except for specific cases like $X = N$ [16]. There can be many solutions to a $2 - (H, N, 1)$ BIBD and not all of them have favorable properties for CXL pods. For example, some BIBDs are partitionable, i.e., break the CXL pod into disconnected sub-pods.

To guarantee non-partitionability and symmetry, we use a cyclical construction method from mathematical literature [16, 24, 114]. A cyclical construction method arranges elements cyclically, generating blocks via systematic cyclic shifts. Specifically, let H hosts be numbered from 0 to $H - 1$. Given parameters X and N , define an initial block (PD) as an arithmetic progression modulo H . The remaining blocks result from cyclically shifting the initial block by increments:

$$B_0 = \{0, d, 2d, \dots, (N - 1)d\} \mod H,$$

where d is relatively prime to H . Subsequent blocks are:

$$B_i = (B_0 + i) \mod H, \quad \text{for } i = 1, 2, \dots, M - 1.$$

For $X = 8$ and $N = 4$, our PD blocks are $[0, 2, 6, 7]$, $[0, 1, 4, 12]$. For $X = 4$, our block is $[0, 1, 3, 9]$.

5.2.2 Inter-Island Connectivity

The main purpose of inter-island connectivity is to increase MPD reachability for pooling. We use dedicated "external" MPDs and the remaining $X - X_i$ host ports. For $X_i = 5$, every island has 48 external links. For our example with six islands

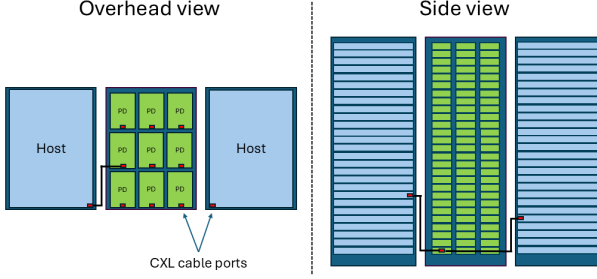


Figure 9: A 3-rack Acadia configuration.

(96 hosts), this means that there are 72 external MPDs, i.e., almost 30% of all MPDs are external.

We find that inter-island connectivity is again a challenging combinatorial design problem. An effective starter heuristic is that every external MPD should connect to hosts from *different* islands. However, we find that we must actively enforce high symmetry. Specifically, each pair of islands must intersect in the same number of MPDs. Finally, any two hosts from different islands should share at most one external MPD to cap worst-case contention. In short, we require uniformity at multiple scales: across islands, across island pairs, and across host pairs.

Our solution to the design problem again splits the task into two levels. At the first level, we find a block design for each external MPD that defines which islands should interconnect. In practice, the block design cannot always be solved exactly, so we fall back to a round-robin heuristic. The second level, assigns hosts from the sampled island to the specific MPD ports. With $X_i = 5$, there are three ports per host. We proceed in three rounds, where in each round every host is used exactly once. We additionally ensure that any two hosts from different islands share at most one external MPD.

5.3 Physical Layout

The physical layout of an Acadia topology is largely driven by CXL cable lengths, cable routing, and constraints of typical datacenter rack layouts. Reasonable cable lengths should be less than 1.5m. To deploy pods with up to 96 servers in a datacenter row, we propose a 3-rack pod designs where pooling devices are placed in the middle rack and compute hosts placed in adjacent racks. Figure 9 sketches this design. The middle rack can hold a varying number of MPDs in each slot, depending on the MPD size (e.g., $N = 4$ vs $N = 8$). The dimension of each standard rack slot is approximately $100 \times 60 \times 5$ cm.

To further optimize cable lengths, we assume compute hosts where CXL edge connectors are located at the front corner of the server chassis closest to the MPD rack, directly on the motherboard. This is a similar requirement to those imposed by the OCP NIC 3.0 specification [92] which also

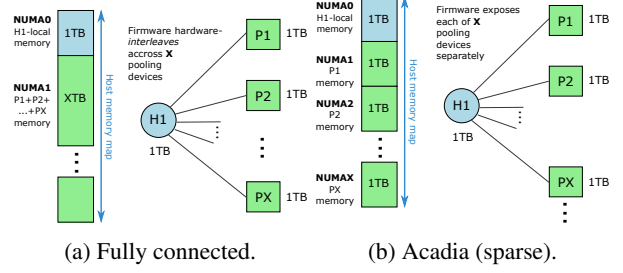


Figure 10: Host view of CXL memory under full interleaving vs. Acadia.

requires PCIe5 ports at the front of the server. Based on existing designs, we assume that CXL ports on the MPD side are in the front-middle of each MPD.

We validate achievable topologies in Section 6.

5.4 Acadia Software Stack

API and exposure. In fully connected pods, firmware configures interleaves CXL address maps at 256B across devices [25], yielding one large pool (Fig. 10a). Acadia disables this interleaving in firmware and exposes each CXL port as a distinct NUMA node (Fig. 10b). This lets software target a specific MPD for capacity balancing and for sharing with peers that attach to the same MPD.

A datacenter control plane (e.g., Borg/Protean-like) assigns host IDs and disseminates the pod topology and per-host MPD set [43, 122].

Pooling policy. Acadia allocates from the *least-loaded* reachable MPD in descending order of available capacity. This reduces allocation failures caused by local hot spots without assuming global defragmentation.

Bandwidth scaling. When one $\times 8$ link is insufficient, Acadia software-interleaves at page granularity (4 kB/2 MB) across multiple MPDs, or uses bandwidth-aware tiering [124]. We prefer the fewest MPDs that meet observed bandwidth to facilitate greedy balancing.

Sharing within an island. Any two hosts in an island share at least one MPD. Acadia places per-MPD input queues there and uses a lightweight poll loop per queue. RPCs use these queues; shuffles and similar collectives write to all targets' queues, then read back. No server forwarding or multi-hop paths are required.

6 Evaluation

Our evaluation of Acadia CXL pod designs is structured into small-scale hardware prototype evaluation, scaled-up simulations, physical layout validation, and cost modeling.

6.1 Evaluation Setup

Hardware We have access to three pre-production MPDs with two $\times 8$ CXL 2.0 ports (*i.e.* $N = 2$). To connect them, we soldered custom risers to route CXL control signals to all ports. We use three systems with an AMD EPYC 9825 Processor, local DDR5-5600 memory, and a 100 Gbit Mellanox CZ5 NIC. The system has 64 lanes, but we connect only 16 lanes (two $\times 8$ lanes) per CPU to form the most basic Acadia island: each pair of systems shares one MPD. All systems additionally connect with a 100 Gbit Arista 7060X series switch and run an updated Ubuntu 24.04.

To measure RPC latency with CXL, we implement a simple RPC by passing a message that contains the RPC parameters from one host to another host and returning a new message containing the return value back to the initiating host. To pass a message, the sender host first writes the message to a MPD, while the receiver host busy polls the MPD to retrieve the message. The message can be a copy of parameters/return values or a pointer to them. To measure the RPC latency with RDMA, we use the `send RDMA` verb to send RPC parameters and to send return values between two hosts. We also measure RPC latency with a user-space networking stack [36].

Simulations Our simulations assume scaled-up Acadia topologies with between two to eight MPD ports (N), though we focus on $N = 4$. We consider host ports (X) between 4 to 16, though we focus on $X = 8$. Our main configuration is Acadia-96 with six islands of 16-hosts each.

Memory pooling simulations play back VM memory demand production traces at CompanyX.³ We collected trace data over a two-week period in December 2024 from thousands of cloud VMs. For each entity, we recorded allocation and deallocation times, resource allocation, and the server hosting the workload.

We assume that when a new VM is launched on a server, it greedily allocates memory from the least utilized MPDs connected to the server (see Section 5.4). A VM may allocate its memory from multiple MPDs for load balancing. The allocated CXL memory is released when VMs terminate. We randomly assign servers in the traces to the servers in the topology. To measure the memory saving from pooling, we replay the traces and record the highest MPD usage across all MPDs throughout the entire simulation period. The highest MPD usage is then used as the capacity for all MPDs. With better pooling, each MPD can have a lower memory capacity, thus reducing the overall memory cost.

Physical layout model We validate the physical realization of a given Acadia topology within the 3-rack layout (Section 5) under CXL cable length constraints by modeling server and MPD placement as a satisfiability (SAT) problem. Each potential host and MPD location within the racks is modeled as a 3-D coordinate that specifies the location of the ports for each host/MPD slot.

³CompanyX is a large cloud provider. Name anonymized for submission.

Islands	Pod Size	CXL Capex Cost	Cable Len.
1	25	\$1252 / host	0.7 m
4	64	\$1292 / host	0.9 m
6	96	\$1548 / host	1.3 m

Table 1: Common Acadia configurations based $X = 8$ host ports and $N = 4$ MPD ports. The CXL cost includes CXL devices and required cables. The cable length is the known minimum required to realize the Acadia topology within a 3-rack configuration. Increased cable cost is the main reason for increasing costs of Acadia-96.

The cable length limit is modeled as a three-dimensional Manhattan distance between host and MPD ports, and is enforced through constraints on the one-to-one mapping of logical MPD and host pairs – as defined by a given logical topology – to physical locations in the racks. Specifically, if logical MPD i and host j are connected in the topology, any physical pair of MPD α and host β that have a Manhattan distance greater than the cable length must not both be mapped to i and j , *i.e.*, $\neg(i \rightarrow \alpha) \vee \neg(j \rightarrow \beta)$. If a mapping that satisfies the constraints is found, then the mapping should be realizable physically, modulo cabling complexity. We implement the model in PySAT [52] and use MiniSat2.2 to sweep the cable length to find the shortest satisfiable cable constraint.

Cost model DRAM and CXL have small power impacts [127], so we focus on purchase cost (Capex). To model the cost overhead of CXL, we assume a hyperscale scenario where a large number of racks adopt CXL. While the purchase cost of a smaller CXL pod may be lower than a larger CXL pod, a hyperscaler will have to buy proportionally more of the smaller CXL pods. We assume a server purchase cost of \$30k and the estimated cost per MPD from Figure 4. We extrapolate slowdown based on latency sensitivity measurements (Figure 3) to determine the fraction of workloads that can run on CXL with MPDs and switches, respectively. We consider a 10% slowdown to be tolerable, which leads us to assume 65% of DRAM can come from MPDs and 35% of DRAM can be provisioned with switches, respectively. For switches, we assume that we retain some local CXL to fulfill the remaining capacity need.

6.2 Hardware Prototype

We first briefly describe our current MPD’s characteristics and then evaluate our three-host island prototype. We show that we can achieve low-latency RPCs, motivate why islands are necessary, and quantify bandwidth gains for island-wide collective communication. Our hardware prototype is essentially a scaled-down version of a current top-line server: we use only two $\times 8$ CXL ports where a current server uses eight $\times 8$ CXL ports. Similarly, we use a single 100 Gbit NIC where a current server may use a 400 Gbit NIC. In relative compari-

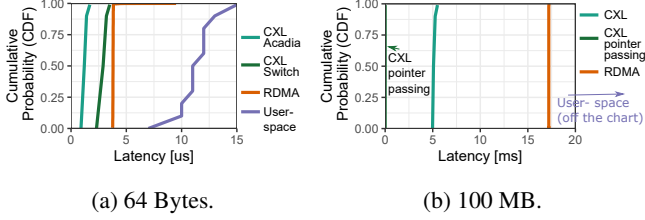


Figure 11: Distribution of RPC round-trip latency for small and large messages. CXL’s latency is $3.3\times$ and $1.5\times$ lower than RDMA, and $10\times$ lower than user-space networking.

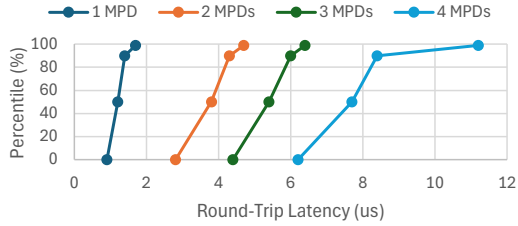


Figure 12: Round-trip RPC latency with messages being transmitted through a varying number of MPDs. Within an island in Acadia, passing messages between any pair of hosts only go through a single MPD.

son, we scaled down along both directions by a factor of four – and thus remain comparable.

MPD hardware characteristics. We measure each $\times 8$ CXL link to provide 24.7 GB/s read-only bandwidth and 22.5 GB/s write-only bandwidth. However, for mixed workloads 1:1 read-write bandwidth increases only to 28.8 GB/s when a single host is active. This is less than expected since the CXL link itself is fully bidirectional. When both hosts are active, each host’s bandwidth maxes out at 22.1 GB/s.

We believe the cause to be a device-internal bottleneck. This bottleneck is likely in the DDR5 memory controller which would need to serve 57.6 GB/s. It is possible that our MPD prototypes operate DIMMs are lower speed, causing a DDR5 bottleneck. We expect that production MPDs will overcome this limitation.

Small RPCs We measure the round-trip latency of RPC where the RPC parameters and return values are both 64 B. With Acadia, given that any pair of hosts within an island has a common MPD, each RPC message is only transmitted through a single MPD.

The results show that the median RPC latency with Acadia is 1.2 μ s (Figure 11a). A CXL switch would have $2.4\times$ higher latency. RDMA has $3.3\times$ higher latency at 3.8 μ s. User-space networking has $9.5\times$ higher latency at over 11 μ s.

We validate Acadia’s island concept by measuring RPC latency when hosts have to forward messages because they do not share a MPD. For example, random or expander topologies with 96 hosts typically require traversing to 3 MPDs. We

measure the round-trip RPC latency with a varying number of intermediate MPDs. Figure 12 shows that transmitting a message through two MPDs increases the median latency from 1.2 μ s to 3.8 μ s, which is comparable to RDMA’s latency.

Large RPCs. We measure 100 MB parameters and 64 B return values. With Acadia, we can pass RPC parameters by value and by reference. For the latter we assume that all parameters are already in the MPD; so there is no need to serialize and copy the parameter.

Figure 11b shows the round-trip RPC latency. When passing by value, CXL achieves 5.1 ms median latency. The median latency with RDMA is $3.3\times$ higher. When passing by reference, CXL has the same RPC latency as the latency with 64 B parameters, which is orders of magnitude lower than passing by value.

Broadcast collectives. The source host connects directly to the two other hosts each via a separate MPD. Thus, the source host writes the data to the two MPDs in parallel and both destination hosts read the data from the corresponding MPD in a pipeline while the source host is still writing. We measure a completion time for broadcasting 32 GB to two hosts at 1.5 s. This means that the MPD indeed serves at full bandwidth to both hosts in parallel. This leads to about a $2\times$ speedup over RDMA.

All-gather collectives. Since the CXL links in the three-host island form a cycle that traverses all three hosts, we use the ring all-gather algorithm to achieve the optimal completion time. A perfectly bidirectional CXL link would allow us to run two ring all-gather in parallel at full bandwidth. However, we cannot fully saturate the CXL links in both directions. The completion time of all-gather with 32 GB shard at each host is 2.9 s. This means that the combined bandwidth of two directions is 22.1 GB/s instead of the expected 28.8 GB/s.

6.3 Scaled-up Simulations

We use the hardware measurements to simulate scaled-up versions of Acadia with 96 hosts.

6.3.1 Memory Pooling Savings

We show that Acadia achieves almost optimal memory pooling savings among MPD topologies with $X = 8$ and $N = 4$. We also compare Acadia to CXL switches which have better reachability than MPDs. However, CXL switches also have higher latency and thus can pool a smaller fraction of CXL capacity. By hosting pooling a larger fraction of memory capacity, Acadia is able to match the pooling savings of CXL switches even under very optimistic assumptions on the CXL switch topology.

Acadia vs. other MPD topologies To show that Acadia achieves near-optimal pooling savings, we compare Acadia to random topologies (*i.e.* Jellyfish [109]) with $X = 8$ and $N = 4$.

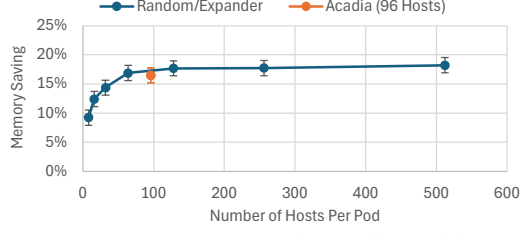


Figure 13: Average memory pooling savings with Acadia and random topologies with a varying number of hosts within a CXL pod. The standard deviation is shown as the error bars.

For random topologies, we vary the number of hosts in the topology. Note that all the topologies have the same MPD cost per host since the host-to-MPD ratio is the same. We find that our random topologies behave like expander topologies (*i.e.* Xpander [46]), and thus report a combined metric.

Figure 13 shows pooling savings for random MPD topologies up to 512 hosts. Note that these are not physically feasible topologies, as copper cable lengths will not scale beyond two racks, and roughly 100 hosts. These large and unrealistic MPD topologies save up to 18% compared to Acadia-96 which saves 16%. In principle, larger topologies allow better reachability and load balancing among hosts with high memory demands. However, the benefits flatten out around 100 hosts which is consistent with the aggregation of peak-to-average demand ratios we observed in Figure 5. This observation validates Acadia’s decision to focus on 96 hosts and stay with practical copper cabling.

Acadia vs. CXL switches CXL switches are subject to similar limitations as MPDs under fully-connected wiring assumptions. Specifically, when every CXL switch must connect to every host (as in prior work), then we cannot connect more than 20 hosts in a pod, because we must at least use 10 ports for CXL devices and 2 ports for management [53]. When pooling across 20 hosts, CXL switches can save 12% of memory, compared to 16% for Acadia-96.

We also simulate an optimistic version of CXL switches which upper bounds their pooling savings. Specifically, if switches adopt a sparsely-connected topology like Acadia and give up on management ports entirely, switches could connect 90 hosts. Since this topology is sparse, it will be less efficient than globally sharing the entire capacity of the connected CXL expansion devices. However, for simplicity we assume that the 90-host switch topology behaves like a global pool. Our simulation shows that the CXL switch topology achieves a 16% memory saving in this scenario, matching the memory saving of Acadia. We find that this optimistic switch topology saves 46% of the pooled memory, but only 35% of all DRAM can be pooled. Acadia saves 25% of pooled memory, while being able to pool 65% of all DRAM.

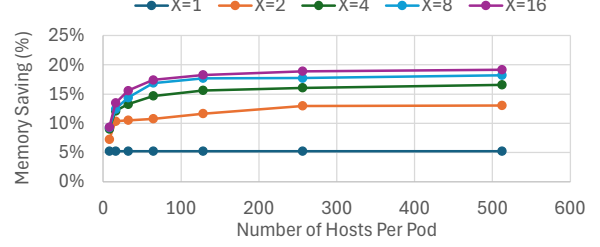


Figure 14: Average memory pooling savings of random MPD topologies with a varying number of hosts and a varying host port count (X).

Sensitivity analysis To show a wide range of memory savings, we consider random topologies. For reference, Acadia-96 behaves similar to a random topology with 64 hosts. Similarly, a variant of Acadia for $X = 4$ would have islands of size 9, giving 36 hosts, comparable to a random topology with 32 hosts. Figure 14 shows memory savings for random topologies from 2 to 512 hosts and $X = 1$ to $X = 16$. Generally, memory savings increase with X . However, $X = 16$ provides marginal benefits over $X = 8$.

We also simulate different number of MPD ports, N . For $N = 2$, pooling savings are minimal. This configuration is not able to reach sufficient numbers of MPDs even with $X = 8$. Conversely, $N = 8$ is significantly more effective than $N = 4$, although we are not aware of $N = 8$ MPDs available today.

6.3.2 Bandwidth-Bound Communication

Acadia provides both low host-to-host communication latency and high bandwidth within each island. We show that Acadia achieves optimal performance for bandwidth-bound communication within each island.

Single active island We simulate the optimal completion time of a uniform all-to-all communication within an island. The optimal completion time is obtained by solving a multi-commodity max flow problem [69] using linear programming. We assume that only one island is active, which allows the active island to also route traffic through inactive islands.

Our simulations show that the all-to-all communication within the active island achieves the optimal bandwidth, in which each host fully saturates its 8 CXL links, including 5 intra-island CXL links and 3 inter-island CXL links. This shows that Acadia can utilize unused link bandwidth outside the island to improve communication performance.

Random traffic However, as Acadia has less inter-island bandwidth, it has lower performance under random communication compared to a random topology. Figure 15 shows that with 10% active hosts, Acadia has 12% lower performance.

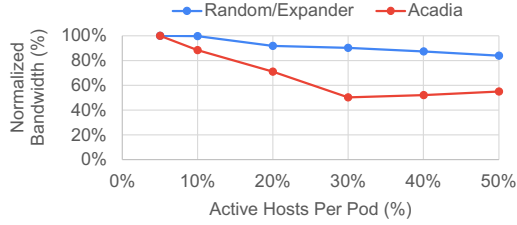


Figure 15: Average normalized bandwidth under random traffic with a varying number of active hosts.

6.4 Physical Layout Validation

To find physical placements for Acadia topologies within a 3-rack system, we attempt solve the physical mapping SAT problem for various cable lengths and determine the length required to realize each topology in Table 1. The cable lengths for which a solution was found within 48 hours of wall-clock time are documented in Table 1. All three Acadia topologies can be realized with cable lengths of 1.3 m or less.

6.5 Cost Comparison

Table 2 summarizes the CXL device Capex and memory pooling cost savings of the 96-host Acadia and the 90-host CXL switch topology. Although Acadia and the CXL switch topology achieve the same memory pooling savings, the switch topology’s device cost is more than twice that of Acadia. Acadia’s cost represents 5% of server Capex, and the switch cost would be 12% of server Capex.

Combining the device Capex and memory pooling savings shows that Acadia reduces the overall server Capex by 3.0% comparing to servers without CXL. This may appear small at first, but this is significant at hyperscale. Also, note that we have not accounted for any savings from data sharing and communication use cases. These other use cases may be able to reduce required CPU core counts, by reducing RPC overheads.

The overall server Capex of the switch topology is 3.3% *higher* than the baseline. Due to the expensive price of switch and the many cables required, the CXL switch topology costs more than it saves in terms of DRAM Capex.

So far, we have assumed a server without CXL expansion as a baseline. Since CXL memory expansion is seeing increasing datacenter adoption, we consider a baseline where the server Capex includes CXL memory expansion. In this scenario, Acadia reduces the server Capex by 5.4%. The key here is that Acadia’s MPDs cost little more than the existing expansion devices. In contrast, the switch topology still increases the server Capex by 0.6%.

7 Related Work

CXL pod topologies. Acadia is, to our knowledge, the first work to compare sparsely connected CXL pod *topologies* and

Topology	Pod Size	CXL Capex	Mem Saving
Expansion	\	\$800 / host	\
Acadia	96	\$1548 / host	16%
Switch	90	\$3460 / host	16%

Table 2: Comparison of CXL device Capex costs and memory pooling cost savings of Acadia and CXL switches.

layouts rather than assuming one wiring pattern. Most prior work and industry decks adopt fully connected MPD pods or switch fabrics and study device properties, not topology tradeoffs across pooling and communication [32, 42, 44, 50, 77, 82, 125]. Typical designs connect every CPU to the same X MPDs so that N -ported MPDs yield pods of size N ; this scales poorly in port count [32, 42, 44, 50, 77, 82, 125].

Scale-up “islands.” Large shared-memory systems (HPE Superdome Flex, SGI UV) compose high-bandwidth islands with longer inter-island paths; Acadia islands mirror this: dense local sharing on a common MPD with sparser inter-island links, aligning with MDC/OpenFAM-style locality-aware programming [2, 3, 57, 58, 108].

Interconnection evolution. Point-to-point device fabrics and switches have traded places across decades: trees to high-radix, low-diameter designs (flattened butterfly, Slim/X-pander), and Clos-like CXL 3.0 fabrics [15, 17, 18, 29, 56, 60–62, 64, 70, 88]. Acadia targets near-term copper-first deployments with sparse MPD bipartite graphs with one-hop sharing while expanding pooling reach. Server-forwarded schemes like BCube [40] are less attractive under CXL latency.

8 Conclusion

Acadia challenges the prevailing assumption that effective CXL pods must employ fully-connected topologies with large pooling devices. Our sparsely-connected topologies are cost-efficient for pooling and pair-wise data sharing, which requires a careful trade-off between competing graph properties. We find that Acadia pods provide benefits to memory-intensive cloud and database workloads. Ultimately, Acadia offers a promising blueprint for CXL memory pooling infrastructures, balancing cost-effectiveness, scalability, and performance.

References

- [1] Pcuo – PCIe®-over-fiber, firefly™ optical cable assembly. <https://www.samtec.com/products/pcuo>. Accessed 2025-09-11.
- [2] SGI UV 300 system reliability, availability and serviceability. White paper, Silicon Graphics International Corp., 2016. Accessed 2025-09-16.

- [3] Hpe superdome flex server architecture and ras. Technical white paper, Hewlett Packard Enterprise, 2021. Accessed 2025-09-16.
- [4] Advanced Micro Devices, Inc. AMD EPYC™ 9005 Processor Architecture Overview. https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/user-guides/58462_amd-epyc-9005-tg-architecture-overview.pdf, Apr. 2025. Specifies up to 64 lanes of CXL 2.0 and examples of CXL-backed memory capacity.
- [5] H. Ahn, S. Kim, Y. Park, W. Han, S. Ahn, T. Tran, B. Ramesh, H. Subramoni, and D. K. Panda. Mpi allgather utilizing cxl shared memory pool in multi-node computing systems. In *2024 IEEE International Conference on Big Data (BigData)*, pages 332–337. IEEE, 2024.
- [6] M. Ahn, T. Willhalm, N. May, D. Lee, S. M. Desai, D. Booss, J. Kim, N. Singh, D. Ritter, and O. Rebolz. An examination of cxl memory use cases for in-memory database management systems using sap hana. *Proceedings of the VLDB Endowment*, 17(12):3827–3840, 2024.
- [7] Amazon Web Services. Adding aurora replicas to a db cluster, 2025. Aurora DB cluster supports up to 15 read replicas plus one writer.
- [8] Amazon Web Services. Quotas and limits in amazon redshift, 2025. Maximum 128 RA3 nodes per cluster.
- [9] Arm Limited. Overview of Neoverse V2 Reference Design. <https://developer.arm.com/documentation/102759/latest/Overview-of-Neoverse-V2-reference-design>, 2024. States eight CML links support CML_SMP and CXL 2.0 for connections to accelerators/memory devices.
- [10] I. Astera Labs. Aries pcie®/cxl® smart cable modules™. <https://www.asteralabs.com/products/aries-smart-cable-modules/>, 2024. Product Brief.
- [11] D. F. Bacon, C. Binnig, D. Patterson, and M. Seltzer. Hardware support for cloud database systems in the post-moore’s law era (dagstuhl seminar 24162). *Dagstuhl Reports*, 14(4):54–84, 2024.
- [12] L. A. Barroso, J. Clidaras, and U. Hözlze. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*. Morgan & Claypool, 3rd edition, 2018. Discusses how hyperscale server component procurement and pricing are typically confidential and governed by non-disclosure agreements.
- [13] A. Baumstark, M. Paradies, K.-U. Sattler, S. Kläbe, and S. Baumann. So far and yet so near-accelerating distributed joins with cxl. In *Proceedings of the 20th International Workshop on Data Management on New Hardware*, pages 1–9, 2024.
- [14] D. S. Berger, D. Ernst, H. Li, P. Zardoshti, M. Shah, S. Rajadnya, S. Lee, L. Hsu, I. Agarwal, M. D. Hill, et al. Design tradeoffs in cxl-based memory pools for public cloud platforms. *IEEE Micro*, 43(2):30–38, 2023.
- [15] M. Besta and T. Hoefer. Slim fly: A cost effective low-diameter network topology. In *SC’14: proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 348–359. IEEE, 2014.
- [16] T. Beth, D. Jungnickel, and H. Lenz. *Design Theory*, volume 1. Cambridge University Press, Cambridge, UK, 2nd edition, 1999. Covers constructions of combinatorial designs, including BIBDs using finite projective planes.
- [17] T. Bjerregaard and S. Mahadevan. A survey of research and practices of network-on-chip. *ACM Computing Surveys (CSUR)*, 38(1):1–es, 2006.
- [18] N. Blach, M. Besta, D. De Sensi, J. Domke, H. Harake, S. Li, P. Iff, M. Konieczny, K. Lakhota, A. Kubicek, et al. A {High-Performance} design, implementation, deployment, and evaluation of the slim fly network. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1025–1044, 2024.
- [19] R. C. Bose. On balanced incomplete block designs. *Annals of Human Genetics*, 9:353–399, 1939.
- [20] P. Chauhan, C. Petersen, B. Morris, and J. Glisse. Hyperscale tiered memory expander specification for compute express link. Available at <https://www.opencompute.org/documents/hyperscale-tiered-memory-expander-specification-for-2023>. Open Compute Project, Revision 1, Effective October 27, 2023.
- [21] Citus Data (Microsoft). Cluster management, 2025. Operational guidance for scaling Postgres with Citus.
- [22] L. Clarke, I. Glendinning, and R. Hempel. The mpi message passing interface standard. In *Programming Environments for Massively Parallel Distributed Systems: Working Conference of the IFIP WG 10.3, April 25–29, 1994*, pages 213–218. Springer, 1994.

- [23] D. Cohen, S. Cohen, D. Naor, D. Waddington, and M. Hershcovitch. Dictionary based cache line compression. In *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*, pages 8–14, 2024.
- [24] C. J. Colbourn and J. H. Dinitz, editors. *Handbook of Combinatorial Designs*. Chapman and Hall/CRC, 2nd edition, 2006.
- [25] Compute Express Link Consortium. Compute Express Link (CXL) Specification, Revision 2.0, nov 2020. Accessed: 2025-03-11.
- [26] D. Corneil and R. Mathon. Algorithmic techniques for the generation and analysis of strongly regular graphs and other combinatorial configurations. In *Annals of Discrete Mathematics*, volume 2, pages 1–32. Elsevier, 1978.
- [27] I. Corporation. Intel Flat Memory Mode: Expanding System Memory with CXL. <https://www.intel.com/content/www/us/en/products/docs/xeon-6-product-brief.html>, 2024.
- [28] M. Cowan, S. Maleki, M. Musuvathi, O. Saarikivi, and Y. Xiong. Mscclang: Microsoft collective communication language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 502–514, 2023.
- [29] W. J. Dally. Performance analysis of k-ary n-cube interconnection networks. *IEEE transactions on Computers*, 39(6):775–785, 1990.
- [30] D. Das Sharma, R. Blankenship, and D. Berger. An introduction to the compute express link (cxl) interconnect. *ACM Computing Surveys*, 56(11):1–37, 2024.
- [31] P. Duraisamy, W. Xu, S. Hare, R. Rajwar, D. Culler, Z. Xu, J. Fan, C. Kennelly, B. McCloskey, D. Mijailovic, B. Morris, C. Mukherjee, J. Ren, G. Thelen, P. Turner, C. Villavieja, P. Ranganathan, and A. Vahdat. Towards an Adaptable Systems Architecture for Memory Tiering at Warehouse-Scale. In *ASPLOS*, 2023.
- [32] M. El-Batal. Seagate Composable Memory Appliance (CMA) Architecture. <https://www.youtube.com/watch?v=KCgE0WvjX10>, June 2024.
- [33] H. P. Enterprise. Hpe 128 gb (1x128b) dual-rank x4 ddr5-6400 cas ec8 registered smart memory kit. HPE product listing (HPE Store, US), 2025. <https://buy.hpe.com/us/en/options/enterprise-memory-options/server-memory/server-memory/hpe-ddr5-smart-memory/p/1013630793>.
- [34] M. Ewais and P. Chow. Disaggregated memory in the datacenter: A survey. *IEEE Access*, 11:20688–20712, 2023.
- [35] R. A. Fisher and F. Yates. The construction of balanced incomplete block designs. *Annals of Human Genetics*, 9:30–43, 1938.
- [36] J. Fried, G. I. Chaudhry, E. Saurez, E. Choukse, Í. Goiri, S. Elnikety, R. Fonseca, and A. Belay. Making kernel bypass practical for the cloud with junction. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 55–73, 2024.
- [37] E. Giles and P. Varman. Acid support for compute express link memory transactions. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 982–995. IEEE, 2024.
- [38] Google Cloud. Create a production instance (cloud bigtable sample), 2025. Sample shows minimum of 3 serve nodes for production instances.
- [39] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin. Efficient memory disaggregation with infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 649–667, 2017.
- [40] C. Guo, G. Lu, D. Li, H. Wu, X. Zhang, Y. Shi, C. Tian, Y. Zhang, and S. Lu. Bcube: a high performance, server-centric network architecture for modular data centers. *SIGCOMM Comput. Commun. Rev.*, 39(4):63–74, aug 2009.
- [41] Y. Guo and G. Li. A cxl-powered database system: Opportunities and challenges. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 5593–5604. IEEE, 2024.
- [42] M. Ha, J. Ryu, J. Choi, K. Ko, S. Kim, S. Hyun, D. Moon, B. Koh, H. Lee, M. Kim, et al. Dynamic capacity service for improving cxl pooled memory efficiency. *IEEE Micro*, 43(2):39–47, 2023.
- [43] O. Hadary, L. Marshall, I. Menache, A. Pan, E. E. Greff, D. Dion, S. Dorminey, S. Joshi, Y. Chen, M. Russinovich, and T. Moscibroda. Protean:{VM} allocation service at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 845–861, 2020.
- [44] D. Hawkins and M. Bromage. Design Considerations for CXL Device Hardware Coherency (HDM-DB). <https://www.youtube.com/watch?v=2ktM7dPcmqI>, 2024.

- [45] Hewlett Packard Enterprise. Hpe 256 gb (1x256 gb) quad rank x4 ddr5-6400 ec8 registered 3ds smart memory kit. Technical report, Hewlett Packard Enterprise, 2025. Product data sheet for 3DS DDR5-6400 256 GB Smart Memory module.
- [46] S. Hoory, N. Linial, and A. Wigderson. Expander graphs and their applications. *Bulletin of the American Mathematical Society*, 43(4):439–561, 2006.
- [47] T. Huang, Y. Liang, S. Yu, and K. Chen. Txcocket: an innovative solution for efficient cross-node data transmission enabled by cxl-based shared memory. *CCF Transactions on High Performance Computing*, January 2025. Regular Paper, Published: 22 January 2025.
- [48] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. Zookeeper: Wait-free coordination for internet-scale systems. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 145–158. USENIX Association, 2010.
- [49] D. Huye, Y. Shkuro, and R. R. Sambasivan. Lifting the veil on meta’s microservice architecture: Analyses of topology and request workflows. In *2023 USENIX Annual Technical Conference (USENIX ATC ’23)*, 2023.
- [50] R. Hyatt. The quest for bandwidth and capacity: Memory edition, 2023. https://www.hpcuserforum.com/wp-content/uploads/2023/09/Ronen-Hyatt_UnifabriX_The-Quest-for-Bandwidth-and-Capacity-Memory-Edition-Sept-2023-HPC-UF.pdf.
- [51] R. Hyatt. “cxl and memory pools: State of the union”. FMS24: BMKT-102-1: Memory Markets, Future of Memory and Storage 2024, Aug. 2024. Slide deck. UnifabriX. DOI/URL: https://files.futurememorystorage.com/proceedings/2024/20240806_BMKT-102-1_Hyatt.pdf.
- [52] A. Ignatiev, Z. L. Tan, and C. Karamanos. Towards universally accessible SAT technology. In *SAT*, pages 4:1–4:11, 2024.
- [53] J. Jiang. CXL Switch for Scalable & Composable Memory Pooling/Sharing. FMS presentation available at <https://www.xconn-tech.com/products>, 2024.
- [54] A. Kalia, M. Kaminsky, and D. G. Andersen. Data-center rpcs can be general and fast. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’19)*, 2019.
- [55] M. Kalodrich. New supermicro x14 systems, 2024. Accessed: 2024-12-18.
- [56] J. Kao. CXL 2.0 Switch for a Composable Memory System. https://computeexpresslink.org/wp-content/uploads/2024/09/Xconn_CXL-2.0-Switch-for-a-Composable-Memory-System_FMS-2024_FINAL.pdf, October 2024.
- [57] K. Keeton. Memory-driven computing. In *Proceedings of FAST ’17: 15th USENIX Conference on File and Storage Technologies*, Santa Clara, CA, Feb. 2017. USENIX Association.
- [58] K. Keeton, S. Singhal, and M. A. Raymond. The openfam api: A programming model for disaggregated persistent memory. In S. Pophale, N. S. V. Imam, F. Aderholdt, and M. G. Venkata, editors, *OpenSHMEM and Related Technologies. OpenSHMEM in the Era of Extreme Heterogeneity*, volume 11283 of *Lecture Notes in Computer Science*, pages 70–89. Springer, 2019.
- [59] P. Kennedy. Lenovo has a cxl memory monster with 128x 128gb ddr5 dimms, 2024. Accessed: 2024-12-18.
- [60] J. Kim, W. J. Dally, and D. Abts. Flattened butterfly: a cost-efficient topology for high-radix networks. In *Proceedings of the 34th annual international symposium on Computer architecture*, pages 126–137, 2007.
- [61] J. Kim, W. J. Dally, S. Scott, and D. Abts. Technology-driven, highly-scalable dragonfly topology. *ACM SIGARCH Computer Architecture News*, 36(3):77–88, 2008.
- [62] J. Kim, W. J. Dally, B. Towles, and A. K. Gupta. Microarchitecture of a high radix router. In *32nd International Symposium on Computer Architecture (ISCA’05)*, pages 420–431. IEEE, 2005.
- [63] A. Labs. Leo cxl smart memory controllers. Available at <https://www.asteralabs.com/products/leo-cxl-smart-memory-controllers/>, December 2023. Product Brief.
- [64] K. Lakhotia, M. Besta, L. Monroe, K. Isham, P. Iff, T. Hoefler, and F. Petrini. Polarfly: a cost-effective and flexible low-diameter topology. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.
- [65] L. Lamport. The part-time parliament. *ACM Transactions on Computer Systems (TOCS)*, 16(2):133–169, May 1998.
- [66] D. Lee and O. Mutlu. Dram scaling challenges and solutions. *IEEE Micro*, 42(2):14–25, 2022.
- [67] D. Lee, T. Willhalm, M. Ahn, S. Mutalik Desai, D. Booss, N. Singh, D. Ritter, J. Kim, and O. Rebholz.

- Elastic use of far memory for in-memory database management systems. In *Proceedings of the 19th International Workshop on Data Management on New Hardware*, pages 35–43, 2023.
- [68] T. Lee, S. K. Monga, C. Min, and Y. I. Eom. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 17–34, 2023.
- [69] T. Leighton and S. Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *J. ACM*, 46(6):787–832, Nov. 1999.
- [70] C. E. Leiserson. Fat-trees: Universal networks for hardware-efficient supercomputing. *IEEE transactions on Computers*, 100(10):892–901, 1985.
- [71] Lenovo. Lenovo thinksystem sr860 v3 server, 2024. Accessed: 2024-12-18.
- [72] Lenovo. Implementing cxl memory on linux on thinksystem v4 servers. White paper, 2025.
- [73] Lenovo. Thinksystem memory summary: 256 gb truddr5 3ds registered dimm (ddr5-6400). Technical report, Lenovo Press, July 2025. Memory summary listing 256 GB 3DS TruDDR5 RDIMM for ThinkSystem servers.
- [74] Lenovo Press. Sap hana solution brief, 2025. Example configurations up to 94 nodes for BW/data-mart workloads.
- [75] A. Lerner and G. Alonso. Cxl and the return of scale-up database engines. *arXiv preprint arXiv:2401.01150*, 2024.
- [76] P. Levis, K. Lin, and A. Tai. A case against cxl memory pooling. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, pages 18–24, 2023.
- [77] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *ASPLOS*, 2023.
- [78] B. Liskov and J. Cowling. Viewstamped replication revisited. Technical Report MIT-CSAIL-TR-2012-021, MIT, July 2012.
- [79] J. Liu, H. Hadian, Y. Wang, D. S. Berger, M. Nguyen, X. Jian, S. H. Noh, and H. Li. Systematic CXL memory characterization and performance analysis at scale. In *Proceedings of the 2025 ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS’25)*, 2025.
- [80] N. Luehr. Nvidia collective communications library (nccl). <https://developer.nvidia.com/nccl>, 2016. Accessed 1/1/2025.
- [81] T. Ma, Z. Liu, C. Wei, J. Huang, Y. Zhuo, H. Li, N. Zhang, Y. Guan, D. Niu, M. Zhang, et al. {HydraRPC}:{RPC} in the {CXL} era. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 387–395, 2024.
- [82] G. Mackey. You don’t know ‘jack’: Cxl fabric orchestration and management. Available at https://files.futurememorystorage.com/proceedings/2024/20240806_CXLT-102-1_Mackey.pdf, 2024. Presented by Jackrabbit Labs.
- [83] S. Mahar, E. Hajyjasini, S. Lee, Z. Zhang, M. Shen, and S. Swanson. Telepathic datacenters: Fast rpcs using shared cxl memory, 2024.
- [84] I. Marvell Technology. Structera a 2504 memory-expansion controller. Available at <https://www.marvell.com/content/dam/marvell/en/public-collateral/assets/marvell-structera-a-2504-near-memory-accelerator-product-brief.pdf>, 2024. Product Brief, P/N MV-SLA25041-A0-HF350AA-C000.
- [85] MemVerge. Memverge unveils world’s first cxl-based multi-server shared memory at isc. Press release, May 2023. International Supercomputing Conference (ISC), Hamburg, Germany.
- [86] Microsoft. Hyperscale secondary replicas—azure sql database, 2025. Up to 30 named replicas per primary.
- [87] MongoDB, Inc. Replica set members, 2025. Up to 50 members, 7 voting; common 3–7 voting members.
- [88] D. Moore and D. D. Sharma. CXL 3.0: Enabling composable systems with expanded fabric capabilities. https://computeexpresslink.org/wp-content/uploads/2023/12/CXL_3.0-Webinar_FINAL.pdf, October 2022.
- [89] B. Munger, K. Wilcox, J. Sniderman, C. Tung, B. Johnson, R. Schreiber, C. Henrion, K. Gillespie, T. Burd, H. Fair, D. Johnson, J. White, S. McLelland, S. Bakke, J. Olson, R. McCracken, M. Pickett, A. Horiuchi, H. Nguyen, and T. H. Jackson. “zen 4”: The amd 5nm 5.7ghz x86-64 microprocessor core. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 38–39, 2023.
- [90] O. Mutlu. Memory scaling: A systems architecture perspective. *International Memory Workshop (IMW)*, 2013.

- [91] A. Name. *Investigating States and Logged Contents in CXL Memory*. PhD thesis, University Name, 2025.
- [92] O. N. s. OCP Server Workgroup. *OCP NIC 3.0 Design Specification Version 1.5.0 - Release*, September 2024. Available at <https://drive.google.com/file/d/1z7MU6Lpu8xQ19rprxHrL1bjaxtx106iV/view>.
- [93] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX annual technical conference (USENIX ATC 14)*, pages 305–319, 2014.
- [94] Oracle. *Mysql innodb cluster—introduction*, 2025. Production clusters require at least three instances.
- [95] A. Pourhabibi, M. Sutherland, A. Daglis, and B. Falsafi. Cerebro: Evading the rpc tax in datacenters. In *Proceedings of MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, Virtual Event, Greece, 2021. ACM.
- [96] A. Rack. *Gnrd8-2l2t preliminary ceb specifications*, 2024. Accessed: 2024-12-18.
- [97] D. Raghavan, P. Levis, M. Zaharia, and I. Zhang. Break-fast of champions: Towards zero-copy serialization with nic scatter-gather. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21)*. ACM, 2021.
- [98] Redis Ltd. *Scale with redis cluster—recommended topologies*, 2025. Recommends 6-node production clusters (3 masters + 3 replicas).
- [99] I. Samtec. *Successful pcie interconnect guidelines*, 2021. Available at https://blog.samtec.com/wp-content/uploads/2021/04/04_15_2021_successful_PCIE_interconnect_guidelines.pdf. Accessed March 11, 2025.
- [100] SAP SE. *Sap hana tailored data center integration—frequently asked questions*, 2018. FAQ repeating the 16 worker-node limit for TDI.
- [101] SAP SE. *Understanding the maximum number of nodes in sap hana scale-out (kba 3557729)*, 2024. States HANA TDI scale-out is limited to 16 worker nodes.
- [102] S. Scargall. *Cxl server buyer’s guide: A complete list of ga platforms (updated 2025)*. Blog post, June 2025.
- [103] Seagate Technology Team Member(s). *Composable memory appliance (cma) base specification*. Specification, Open Compute Project, Nov. 2024. Seagate contribution to OCP CMS; the CFM spec references this Base Spec.
- [104] K. Seemakhupt, B. E. Stephens, S. Khan, S. Liu, H. Wassel, S. H. Yeganeh, A. C. Snoeren, A. Krishnamurthy, D. E. Culler, and H. M. Levy. A cloud-scale characterization of remote procedure calls. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, pages 1–17, Koblenz, Germany, 2023. ACM.
- [105] S. Sha, C. Li, Y. Luo, X. Wang, and Z. Wang. vTMM: Tiered Memory Management for Virtual Machines. In *EuroSys*, 2023.
- [106] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh. {TACCL}: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, 2023.
- [107] S. S. Shrikhande. A survey of balanced incomplete block designs. *Journal of Combinatorial Theory, Series A*, 15(3):271–291, 1973.
- [108] S. Singhal, C. R. Crasta, M. Abdulla, F. Barmawer, D. Emberson, R. Ahobala, et al. Openfam: A library for programming disaggregated memory. In S. Poole, O. Hernandez, M. Baker, and T. Curtis, editors, *OpenSHMEM and Related Technologies. OpenSHMEM in the Era of Exascale and Smart Networks*, volume 13159 of *Lecture Notes in Computer Science*, pages 21–38. Springer, 2022.
- [109] A. Singla, C.-Y. Hong, L. Popa, and P. B. Godfrey. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, pages 225–238, San Jose, CA, apr 2012. USENIX Association.
- [110] SK hynix. *Fms 2025: Sk hynix presents future of ai and storage through innovative tech showcase*. <https://news.skhynix.com/future-of-memory-and-storage-2025/>, Aug. 2025. Press Release.
- [111] A. B. Smith and J. L. Walrand. Integrated circuit yield and cost analysis: A critical area approach. In *Proceedings of the IEEE International Conference on VLSI Design*, pages 115–120. IEEE, 2005.
- [112] P. Solutions. *Cxl memory expansion servers from penguin solutions*. Product page / datasheet, 2025.
- [113] A. Sriraman and A. Dhanotia. Accelerometer: Understanding acceleration opportunities for data center overheads at hyperscale. In *Proceedings of the 25th International Conference on Architectural Support for*

- [114] D. R. Stinson. *Combinatorial Designs: Constructions and Analysis*. Springer, 2004.
- [115] J. Suetterlein, J. Manzano, and A. Marquez. Synchronization for cxl based memory. In *Proceedings of the International Symposium on Memory Systems, MEMSYS '24*, page 178–185, New York, NY, USA, 2024. Association for Computing Machinery.
- [116] Super Micro Computer, Inc. Storage superserver ssg-121e-ne3x12r datasheet, 2025. Datasheet PDF.
- [117] K. Systems. Kr2280v3 platform intel amd, 2024. Accessed: 2024-12-18.
- [118] D. Technologies. Cxl-supported dell poweredge platforms. Dell InfoHub → “CXL Memory Expanded and Optimized for Dell PowerEdge Servers”, 2025.
- [119] M. Technology. Micron 128 gb ddr5-5600 rdimm 2rx4 cl46 module. Product specification and MSRP listing at Crucial.com, 2025. <https://www.crucial.com/memory/server-ddr5/mtc40f2047s1rc56br>.
- [120] S. Telian. Pcie gen5 signal integrity implementation. In *DesignCon 2023*, 2023. Available at https://siguys.com/wp-content/uploads/2023/02/DesCon23_PAPER_Track7_PCIeGen5SignalIntegrityImplementation_Telian.pdf. Accessed March 11, 2025.
- [121] Y. Tian. Project gismo: Global i/o-free shared memory objects. Presented at Flash Memory Summit (FMS) 2023, <https://memverge.com/wp-content/uploads/MemVerge-Gismo-FMS2023.pdf>, 2023.
- [122] M. Tirmazi, A. Barker, N. Deng, M. E. Haque, Z. G. Qin, S. Hand, M. Harchol-Balter, and J. Wilkes. Borg: the Next Generation. In *Proceedings of the fifteenth European conference on computer systems*, pages 1–14, 2020.
- [123] A. Valadarsky, G. Shahaf, M. Dinitz, and M. Schapira. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '16)*, pages 205–219, Heidelberg, Germany, dec 2016. ACM.
- [124] M. Vuppapapati and R. Agarwal. Tiered memory management: Access latency is the key! In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 79–94, 2024.
- [125] M. Wagh and R. Sodke. Compute express link 2.0 specification: Memory pooling. Available at <https://computeexpresslink.org/wp-content/uploads/2023/12/CXL-2.0-Memory-Pooling.pdf>, 2021. Presented at the CXL Consortium.
- [126] J. Wahlgren, M. Gokhale, and I. B. Peng. Evaluating emerging cxl-enabled memory pooling for hpc systems. In *2022 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*, pages 11–20. IEEE, 2022.
- [127] J. Wang, D. S. Berger, F. Kazhamiaka, C. Irvine, C. Zhang, E. Choukse, K. Frost, R. Fonseca, B. Warrior, C. Bansal, et al. Designing cloud servers for lower carbon. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 452–470. IEEE, 2024.
- [128] J. Wang and Q. Zhang. Disaggregated database systems. In *Companion of the 2023 International Conference on Management of Data*, pages 37–44, 2023.
- [129] M. Weisgut, D. Ritter, P. Tözün, L. Benson, and T. Rabl. Cxl memory performance for in-memory data processing. *Proceedings of the VLDB Endowment*, 18(9):3119–3133, 2025.
- [130] XConn. XC50256 CXL 2.0 Switch Chip. <https://www.xconn-tech.com/products>, 2024.
- [131] L. Xiang, Z. Lin, W. Deng, H. Lu, J. Rao, Y. Yuan, and R. Wang. Nomad: {Non-Exclusive} memory tiering via transactional page migration. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 19–35, 2024.
- [132] C. Xu, Y. Liu, Z. Li, Q. Chen, H. Zhao, D. Zeng, Q. Peng, X. Wu, H. Zhao, S. Fu, and M. Guo. FaaS-Mem: Improving Memory Efficiency of Serverless Computing with Memory Pool Architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 331–348, 2024.
- [133] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, pages 347–356, 2019.
- [134] J. Zhang, X. Chen, Y. Zhang, and Z. Wang. Dmrpc: Disaggregated memory-aware datacenter rpc for data-intensive applications. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3796–3809. IEEE, 2024.

- [135] M. Zhang, T. Ma, J. Hua, Z. Liu, K. Chen, N. Ding, F. Du, J. Jiang, T. Ma, and Y. Wu. Partial failure resilient memory management system for (cxl-based) distributed shared memory. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 658–674, New York, NY, USA, 2023. Association for Computing Machinery.
- [136] Y. Zhong, D. S. Berger, C. Waldspurger, R. Wee, I. Agarwal, R. Agarwal, F. Hady, K. Kumar, M. D. Hill, M. Chowdhury, et al. Managing memory tiers with {CXL} in virtualized environments. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 37–56, 2024.
- [137] Z. Zhu, N. Ni, Y. Huang, Y. Sun, Z. Jia, N. S. Kim, and E. Witchel. Lupin: Tolerating partial failures in a cxl pod. In *Proceedings of the 2nd Workshop on Disruptive Memory Systems*, pages 41–50, 2024.